

数均用 1~4 个小写字母表示。例如, $a(b(f(a,a),b(f(a,a),f)),f(b(f(a,a),b(f(a,a),f)),f))$ 可以表示为图 11-3 (a) 中形式。

用消除公共表达式的方法可以减少表达式树上的结点, 得到一个图, 如图 11-3 (b) 所示。左图有 21 个点, 而右图只有 7 个点。其表示方法为 $a(b(f(a,4),b(3,f)),f(2,6))$, 其中各个结点按照出现顺序编号为 1, 2, 3, ..., 即编号 k 表示目前为止写下的第 k 个结点。

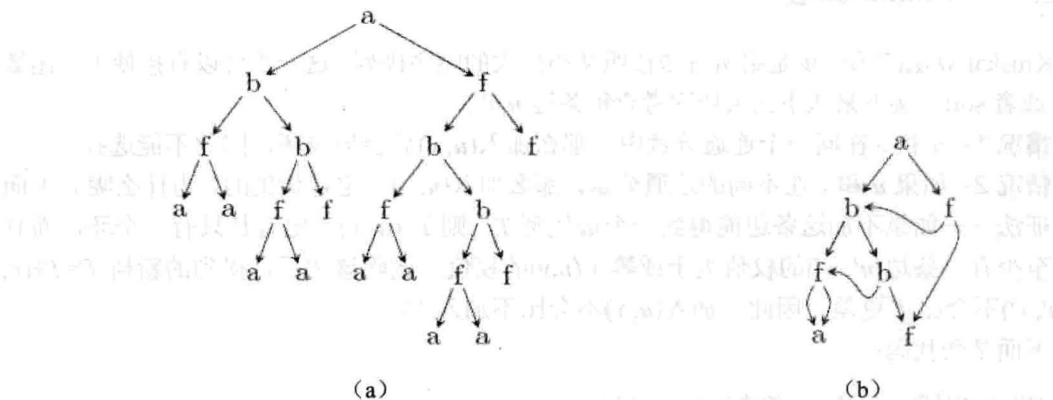


图 11-3 公共表达式消除

输入一个长度不超过 50000 的表达式, 输出一个等价的, 结点最少的图。

【分析】

算法的第一步是构造表达式树。接下来应该怎么做呢? 是否可以用两两比较的方法去掉重复? 比较两棵树的时间复杂度为 $O(n)$ (因为要递归比较二者的所有后代), 再加上二重循环枚举两棵子树, 总时间复杂度高达 $O(n^3)$, 无法承受。此处不仅需要更快地比较两棵树, 还需要更快地查找一棵树是否存在过。

借用第 5 章“集合栈计算机”的思路, 用一个 map 把子树映射成编号 1, 2, ...。这样一来, 子树就可以用根的名字 (字符串) 和左右子结点编号表示。如图 11-4 所示, 用 $(a,0,0)$ 表示根的名字为 a , 且左右子结点均为空 (0 表示不存在) 的子树, 即叶子 a 。可以看到, 下面所有叶子 a 的编号都是 4。再例如, $(b,3,6)$ 就是根的名字为 b , 左右两个子树的编号分别为 3, 6。可以看到, 这样的子树编号均为 5。

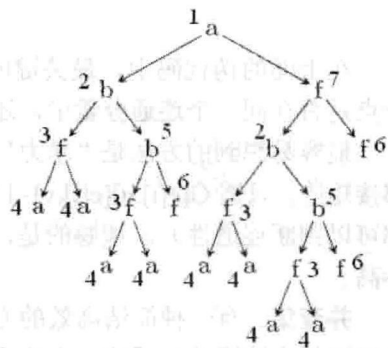


图 11-4 子树编号

这样, 每次判断一棵子树是否出现过只需要在 map 中查找, 总时间复杂度为 $O(n \log n)$ 。

11.2 最小生成树

前面提到过, 在无向图中, 连通且不含圈的图称为树 (Tree)。给定无向图 $G=(V,E)$,

连接 G 中所有点，且边集是 E 的子集的树称为 G 的生成树（Spanning Tree），而权值最小的生成树称为最小生成树（Minimal Spanning Tree, MST）。构造 MST 的算法有很多，最常见的有两个：Kruskal 算法和 Prim 算法。限于篇幅，这里只介绍 Kruskal 算法，它易于编写，而且效率很高。

11.2.1 Kruskal 算法

Kruskal 算法的第一步是给所有边按照从小到大的顺序排列。这一步可以直接使用库函数 `qsort` 或者 `sort`。接下来从小到大依次考查每条边 (u, v) 。

情况 1: u 和 v 在同一个连通分量中，那么加入 (u, v) 后会形成环，因此不能选择。

情况 2: 如果 u 和 v 在不同的连通分量，那么加入 (u, v) 一定是最优的。为什么呢？下面用反证法——如果不加这条边能得到一个最优解 T ，则 $T+(u, v)$ 一定有且只有一个环，而且环中至少有一条边 (u', v') 的权值大于或等于 (u, v) 的权值。删除该边后，得到的新树 $T' = T + (u, v) - (u', v')$ 不会比 T 更差。因此，加入 (u, v) 不会比不加入差。

下面是伪代码：

把所有边排序，记第 i 小的边为 $e[i]$ ($1 \leq i \leq m$)

初始化 MST 为空

初始化连通分量，让每个点自成一个独立的连通分量

```
for(int i = 0; i < m; i++)
```

```
    if( $e[i].u$  和  $e[i].v$  不在同一个连通分量) {
```

```
        把边  $e[i]$  加入 MST
```

```
        合并  $e[i].u$  和  $e[i].v$  所在的连通分量
```

```
    }
```

在上面的伪代码中，最关键的地方在于“连通分量的查询与合并”：需要知道任意两个点是否在同一个连通分量中，还需要合并两个连通分量。

最容易想到的方法是“暴力”——每次“合并”时只在 MST 中加入一条边（如果使用邻接矩阵，只需 $G[e[i].u][e[i].v]=1$ ），而“查询”时直接在 MST 中进行图遍历（DFS 和 BFS 都可以判断连通性）。遗憾的是，这个方法不仅复杂（需要写 DFS 或者 BFS），而且效率不高。

并查集。有一种简洁高效的方法可用来处理这个问题：使用并查集（Union-Find Set）。可以把每个连通分量看成一个集合，该集合包含了连通分量中的所有点。这些点两两连通，而具体的连通方式无关紧要，就好比集合中的元素没有先后顺序之分，只有“属于”和“不属于”的区别。在图中，每个点恰好属于一个连通分量，对应到集合表示中，每个元素恰好属于一个集合。换句话说，图的所有连通分量可以用若干个不相交集来表示。

并查集的精妙之处在于用树来表示集合。例如，若包含点 1, 2, 3, 4, 5, 6 的图有 3 个连通分量 {1,3}、{2,5,6}、{4}，则需要用 3 棵树来表示。这 3 棵树的具体形态无关紧要，只要有一棵树包含 1、3 两个点，一棵树包含 2、5、6 这 3 个点，还有一棵树只包含 4 这一个点即可。规定每棵树的根结点是这棵树所对应的集合的代表元（representative）。

如果把 x 的父结点保存在 $p[x]$ 中（如果 x 没有父结点，则 $p[x]$ 等于 x ），则不难写出“查找结点 x 所在树的根结点”的递归程序：`int find(int x) { p[x] = x ? x : find(p[x]); }`，通俗地讲就是：如果 $p[x]$ 等于 x ，说明 x 本身就是树根，因此返回 x ；否则返回 x 的父结点 $p[x]$ 所在树的树根。

问题来了：在特殊情况下，这棵树可能是一条长长的链。设链的最后一个结点为 x ，则每次执行 `find(x)` 都会遍历整条链，效率十分低下。看上去是个很棘手的问题，其实改进方法很简单。既然每棵树表示的只是一个集合，因此树的形态是无关紧要的，并不需要在“查找”操作之后保持树的形态不变，只要顺便把遍历过的结点都改成树根的子结点，下次查找就会快很多了，如图 11-5 所示。

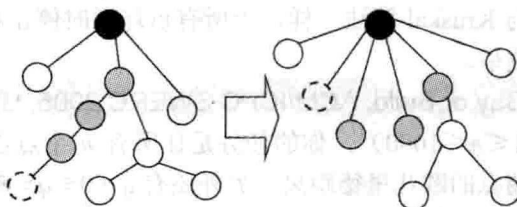


图 11-5 并查集中的路径压缩

这样，Kruskal 算法的完整代码便不难给出了。假设第 i 条边的两个端点序号和权值分别保存在 $u[i]$, $v[i]$ 和 $w[i]$ 中，而排序后第 i 小的边的序号保存在 $r[i]$ 中（这叫做间接排序。排序的关键字是对象的“代号”，而不是对象本身）。

```
int cmp(const int i, const int j) { return w[i] < w[j]; } //间接排序函数
int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); } //并查集的 find
int Kruskal() {
    int ans = 0;
    for(int i = 0; i < n; i++) p[i] = i; //初始化并查集
    for(int i = 0; i < m; i++) r[i] = i; //初始化边序号
    sort(r, r+m, cmp); //给边排序
    for(int i = 0; i < m; i++) {
        int e = r[i]; int x = find(u[e]); int y = find(v[e]); //找出当前边两个端点所在集合编号
        if(x != y) { ans += w[e]; p[x] = y; } //如果在不同集合，合并
    }
    return ans;
}
```

注意， x 和 y 分别是第 e 条边的两个端点所在连通分量的代表元。合并 x 和 y 所在集合可以简单地写成 $p[x]=y$ ，即直接把 x 作为 y 的子结点，则两个树就合并成一棵树了。注意不能写成 $p[u[e]]=p[v[e]]$ ，因为 $u[e]$ 和 $v[e]$ 不一定是树根。并查集的效率非常高，在平摊意义下，`find` 函数的时间复杂度几乎可以看成是常数（而 `union` 显然是常数时间）。

11.2.2 竞赛题目选解

例题 11-2 苗条的生成树 (Slim Span, ACM/ICPC Japan 2007, UVa1395)

给出一个 n ($n \leq 100$) 结点的图，求苗条度（最大边减最小边的值）尽量小的生成树。

【分析】

首先把边按权值从小到大排序。对于一个连续的边集区间 $[L, R]$ ，如果这些边使得 n 个点全部连通，则一定存在一个苗条度不超过 $W[R] - W[L]$ 的生成树（其中 $W[i]$ 表示排序后第 i 条边的权值）。

从小到大枚举 L ，对于每个 L ，从小到大枚举 R ，同时用并查集将新进入 $[L, R]$ 的边两端的点合并成一个集合，与 Kruskal 算法一样。当所有点连通时停止枚举 R ，换下一个 L （并且把 R 重置为 L ）继续枚举。

例题 11-3 买还是建 (Buy or Build, ACM/ICPC SWERC 2005, UVa1151)

平面上有 n 个点 ($1 \leq n \leq 1000$)，你的任务是让所有 n 个点连通。为此，你可以新建一些边，费用等于两个端点的欧几里德距离。另外还有 q ($0 \leq q \leq 8$) 个“套餐”可以购买，如果你购买了第 i 个套餐，该套餐中的所有结点将变得相互连通。第 i 个套餐的花费为 C_i 。如图 11-6 所示，一共有 3 个套餐：

它的最优解是购买套餐 1 和套餐 2，然后手动连接两条边，如图 11-7 所示。

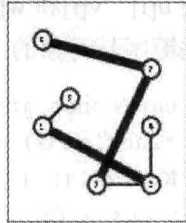
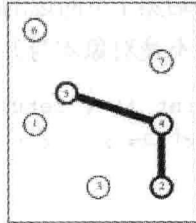
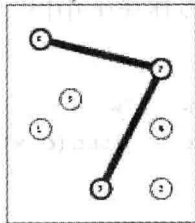
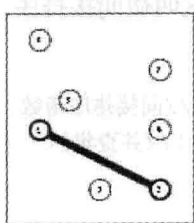


图 11-6 3 个套餐

图 11-7 购买套餐 1、2 并连接边

【分析】

最容易想到的算法是：先枚举购买哪些套餐，把套餐中包含的边的权值设为 0，然后求最小生成树。由于枚举量为 $O(2^q)$ ，给边排序的时间复杂度为 $O(n^2 \log n)$ ，而排序之后每次 Kruskal 算法的时间复杂度为 $O(n^2)$ ，因此总时间复杂度为 $O(2^q n^2 + n^2 \log n)$ ，对于题目的规模来说太大了。

只需一个小小的优化即可降低时间复杂度：先求一次原图（不购买任何套餐）的最小生成树，得到 $n-1$ 条边，然后每次枚举完套餐后只考虑套餐中的边和这 $n-1$ 条边，则枚举套餐之后再求最小生成树时，图上的边已经寥寥无几。

为什么可以这样呢？首先回顾一下，在 Kruskal 算法中，哪些边不会进入最小生成树。答案是：两端已经属于同一个连通分量的边。买了套餐以后，相当于一些边的权变为 0，而对于不在套餐中的每条边 e ，排序在 e 之前的边一个都没少，反而可能多了一些权值为 0 的边，所以在原图 Kruskal 时被“扔掉”的边，在后面的 Kruskal 中也一样会被扔掉。

本题还有一个地方需要说明：因为 Kruskal 在连通分量包含 n 个点时会终止，所以对于